PYTHON ACTIVITY-BASED LEARNING

Case Study 01: University Admission Decision Engine

Dr. RVR NRI Institute of Technology | Python Programming

Student Name: _____ **Roll No:** _____

Branch / Section: _____

Problem Statement

Evaluate applicants using subject eligibility and aggregate marks, then rank eligible applicants. Ties are resolved by higher Mathematics marks, then higher aggregate.

Objectives

• Check each applicant's subject-wise minimum marks.

• Calculate aggregate percentage.

• Select eligible applicants and rank them using fixed tie-break rules.

Assumptions

• Each applicant has marks in Mathematics, Physics and Chemistry, each out of 100.

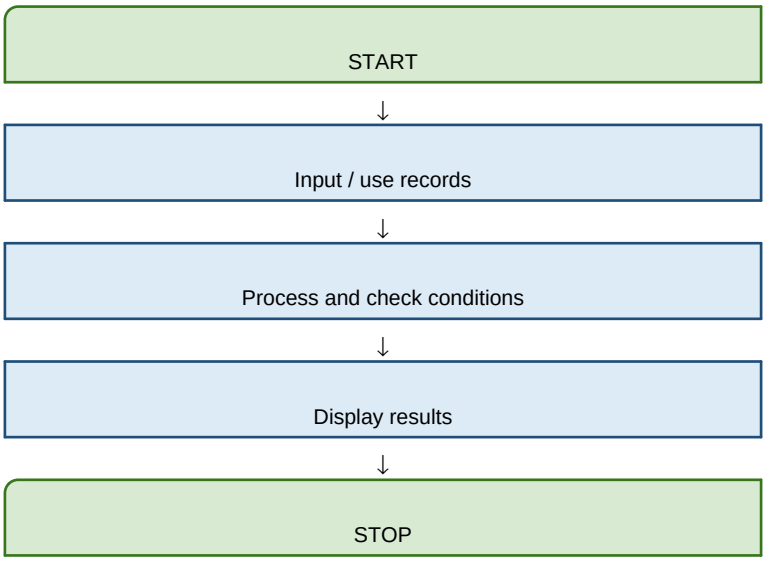
• Minimum mark in every subject is 40 and aggregate percentage must be at least 60%.

• Input data is entered as given in the program; applicant names are unique.

Algorithm

1. Start and define applicant records.
2. For each applicant, calculate the average of the three subject marks.
3. Check whether every subject mark is at least 40 and average is at least 60.
4. Add eligible applicants to a list.
5. Sort eligible applicants by average, then Mathematics marks, both in descending order.
6. Display the ranked list and stop.

Flowchart



Pseudocode

```
START
SET applicants list
FOR each applicant
    CALCULATE average marks
    IF all subject marks >= 40 AND average >= 60
        ADD applicant to eligible list
SORT eligible by average and Mathematics marks (descending)
DISPLAY ranked eligible list
STOP
```

Python Source Code

```
applicants = [
    ("Anu", 85, 78, 80),
    ("Balu", 70, 65, 72),
    ("Chitra", 90, 92, 88),
    ("Deepak", 35, 80, 75)
]

eligible = []

for name, maths, physics, chemistry in applicants:
    average = (maths + physics + chemistry) / 3
    if maths >= 40 and physics >= 40 and chemistry >= 40 and average >= 60:
        eligible.append((name, average, maths))

eligible.sort(key=lambda x: (x[1], x[2]), reverse=True)

print("Rank  Name       Average")
for rank, student in enumerate(eligible, start=1):
    print(rank, student[0], round(student[1], 2))
```

Test Cases

Test input / condition	Expected result
Anu: 85,78,80	Eligible; average 81.00
Deepak: 35,80,75	Not eligible: Mathematics below 40
Balu: 70,65,72	Eligible; average 69.00

Output Screenshot

Sample console output (formatted to show the expected execution result):

```
Rank  Name       Average
1 Chitra 90.0
2 Anu 81.0
3 Balu 69.0
```

Conclusion

The program checks subject-wise and aggregate eligibility and produces a merit list using the specified tie-breaking order.